

Rockchip_Developer_Guide_AEC_Audio_CN

文件标识: RK-KF-YF-xx
发布版本: V1.0.0
日期: 2026-05-06
文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司
Rockchip Electronics Co., Ltd.
地址: 福建省福州市铜盘路软件园A区18号
网址: www.rock-chips.com
客户服务电话: +86-4007-700-590
客户服务传真: +86-591-83951833
客户服务邮箱: fae@rock-chips.com

前言

概述

本文档主要介绍 Rockchip 系列芯片的回采功能的适配及调试。

产品版本

芯片名称	内核版本
RK 系列芯片	Linux 4.19, Linux 5.10, Linux 6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	龚天韵	2026-05-06	初始版本

目录

Rockchip_Developer_Guide_AEC_Audio_CN

- 1、简介
 - 1.1、文档结构拆解
 - 1.2、流程拆解
 - 1.3、名词解释
- 2、软硬件适配
 - 2.1、硬回采+单个CPU DAI
 - dts配置
 - 验证通路
 - 2.2、硬回采+多个CPU DAI
 - 内核配置
 - dts配置
 - 验证通路
 - 2.3、软回采+单个CPU DAI
 - 2.3.1、通用方案（DLP）
 - 内核配置
 - dts配置
 - 验证通路
 - 2.3.2、SAI LP方案
 - dts配置
 - 验证通路
 - 2.4、软回采+多个CPU DAI
 - 2.4.1、通用方案（DLP）
 - 内核配置
 - dts配置
 - 验证通路
 - 2.4.2、SAI LP方案
 - 内核配置
 - dts配置
 - 验证通路
- 3、特殊方案介绍
 - 3.1、HDMI 回采
 - 3.1.1、单HDMI输出
 - 内核配置
 - dts配置
 - 驱动修改
 - 验证通路
 - 3.1.2、SPK+HDMI同时输出
 - 内核配置
 - dts配置
 - hal层适配
 - 验证通路
 - 3.1.3、SPK+HDMI独立输出

4、FAQ

- 4.1、multi声卡没有生成
- 4.2、使用arecord (tinycap) 录音报错
- 4.3、先录音后播放回采声道数据丢失，顺序反过来就正常
- 4.4、回采声道获取到的音频数据听起来有杂音
- 4.5、安卓平台命令录音正常，应用录音报错
- 4.6、音频通路打通后后续调试与适配步骤参考

1、简介

本文档主要介绍回声消除算法中rk平台常见的软硬件方案，客户可根据本文档结合安卓和Linux不同环境下的相关文档做进一步适配与调试

1.1、文档结构拆解

文档整体可分为四部分，分别是简介、常规场景适配介绍、特殊场景适配介绍和FAQ，建议客户先通读简介部分理解1.2小节的流程拆解，然后阅读第二章软硬件适配的开头介绍部分，对回采框架有一个基本的理解后再根据实际情况阅读对应章节，配置过程中有任何的问题请先根据相关章节的提示，结合1.3小节的名词解释表及附录文档链接查阅，然后在FAQ中查找是否有对应问题的解答

1.2、流程拆解

回声消除算法适配分为以下几步：

1. 明确音频方案和硬件拓扑关系（硬回采/软回采，用了哪些codec和对应硬件协议，Linux方案还是安卓方案），确定可行性
2. 根据具体的硬件拓扑关系适配dts，基于基本音频命令（如arecord/aplay，tinycap/tinypay）能正常获取到录音+回采的音频数据
3. 在音频通路打通的基础上依据相关文档进行音频参数标定
4. 适配算法
5. 参数调优

无论是Linux还是安卓平台都要遵循上述流程顺序进行，紧急情况下3、4步可以同时进行此时适配算法只需验证通路是否打通，效果调试仍然依赖于前面步骤的全部完成；其中3-5步都有对应文档可参考，本文档旨在对1-2步总结rk平台的软硬件方案，方便客户更快的理解方案并打通回声消除的基本音频通路

1.3、名词解释

表格中的文档均指 [Rockchip Developer Guide Audio CN V2.4.0.pdf](#)

术语	全称/分类	含义解释	文档/应用场景参考
硬回采	Hard Loopback	硬件回采。 指经由硬件回采 (loopback) 电路, 通过采集模拟信号的方式获取播放的音频数据	对soc而言无特殊配置要求
软回采	Soft Loopback	软件数字回采。 指不经由回采电路在软件层面 (驱动或应用层) 或者芯片内部将播放的数据流直接复制一份作为录音数据流。即 DLP (Digital Loopback) 和 SAI DLP 功能, 可用于特定的麦克风+回采混合场景。	文档“音频数字回采 DLP”章节。支持多种模式, 如 “2CH: 1 Loopback + 1 Mic”。
DLP	Digital Loopback	数字回采模块。 全称 Digital Audio Loopback。它是 Rockchip 驱动中的一个虚拟组件, 用于在软件或固件层面模拟“远场语音交互”中的信号混合。 <u>它可以将播放通道 (Playback) 的数据按特定比例混合到录音通道 (Capture) 中, 用于回声消除 (AEC) 算法的调试和验证。</u>	文档“DLP”章节。支持 <code>single-dai</code> 和 <code>multi-dai</code> 场景, 通过 <code>amixer</code> 控制回采模式。
SAI LP	SAI Loopback	SAI控制器回采。 指在硬件内部通过SAI控制器将发送 (TX) 的数据直接连接到接收 (RX) 端, 无需外部物理连线。用于测试硬件通路是否正常, 常用于硬件自检	文档“loopback sel”章节。支持 <code>single-dai</code> 和 <code>multi-dai</code> 场景, <u>通过 <code>amixer</code> 控制回采模式。</u>
CPU DAI	CPU Digital Audio Interface	系统主控端接口。 在 ASoC 架构中, 指 SoC (CPU) 端的数字音频接口驱动 (如 I2S, PDM, SAI)。它负责通过 DMA 与内存交换数据, 并通过物理总线与外部 Codec 通信。	文档中 Machine 驱动配置的 <code>rockchip,cpu</code> 字段。
Codec DAI	Codec Digital Audio Interface	外部编解码器接口。 指外部音频芯片 (如 ES8388, RT5640, RK817) 端的驱动。它负责将数字信号转换为模拟信号 (播放) 或将模拟信号转换为数字信号 (录音)。	文档中 Machine 驱动配置的 <code>rockchip,codec</code> 字段。
Simple Card	Simple Audio Card	通用声卡驱动。 Linux ASoC 提供的一种标准化 Machine Driver (<code>simple-audio-card</code>)。它通过解析设备树 (DTS) 中的节点属性 (如 <code>simple-audio-card,name</code> , <code>cpu</code> , <code>codec</code> , <code>format</code> , <code>mclk-fs</code> 等) 来自动构建声卡链路, 无需为每个简单的音频组合编写专用的 C 语言驱动代码。	文档“Simple Card”章节。适用于标准的 I2S/PDM + Codec 连接场景, 通过配置 DTS 即可快速点亮声卡。

术语	全称/分类	含义解释	文档/应用场景参考
Multicodec	Multi-Codecs	多 Codec 架构。一种 Machine Driver 驱动模式 (rockchip,multicodecs-card)。允许一个声卡设备同时管理多个 Codec 芯片 (例如: 一个用于播放的功放 + 一个用于录音的 ADC), 或者将 ASRC 模块集成到音频流中。	文档“Multi Codecs Machine Driver”章节。
Multi-Dai	Multi-DAI	多 DAI 组合。一种驱动框架 (rockchip,multi-dais), 用于将多个 DAI (数字音频接口) 组合成一个逻辑声卡。常用于扩展声道数 (如将两个 I2S 合并成 16 声道) 或混合不同类型的接口 (如 I2S + PDM 阵列)。	文档“Combo DAI 驱动开发”及“Multi DAI”实例章节。

2、软硬件适配

回采方案的软硬件配置主要从两个方面考虑，一是**软硬回采的区分**，二是**回采与录音所经由的硬件协议 (即cpu dai)**，对应的回采方案如下表所示：

	单个cpu dai (如i2s)	多个cpu dai (如pdm+i2s或 i2s0+i2s1)
硬回采	simple-card or multicodec	multicodec + multi-dai
软回采	(simple-card or multicodec) + (dlp or sai-lp)	multicodec + multi-dai+ (dlp or sai-lp)

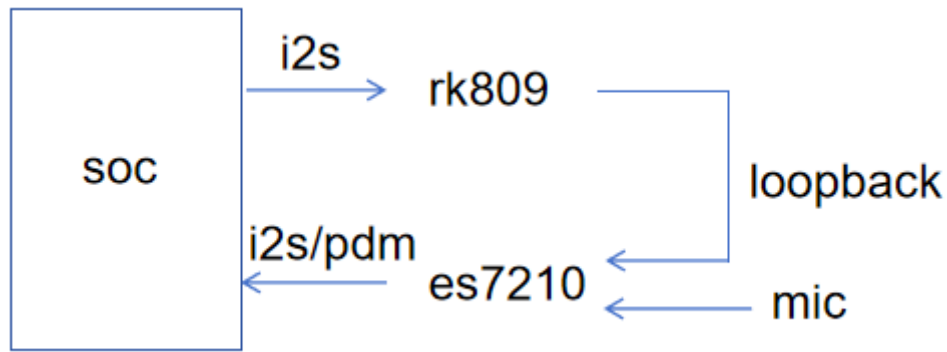
其中**软回采又可以细分为dlp和sai lp**，其区别在于前者是通过驱动软件实现的回采而后者是**通过芯片内部的sai接口实现**，目前后者只能在rk3576等使用sai接口的平台上使用，无论对于任何方案来说其标准验证方法都是：

- 1. 适配dts，cat proc/asound/cards检查声卡有无生成
- 2. 通过amixer (安卓是tinymix) 设置好**声卡录音播放回采所需的控件**
- 3. 通过aplay (安卓是tinypay) 验证播放声卡播放声音是否正常
- 4. 在aplay正常播放的同时arecord (安卓是tinycap) 获取录音+回采的wav文件，这一步骤建议录音的采样率要和播放**同采样率** (比如都是48k)
- 5. 将录到的wav文件pull出来，用音频分析软件 (如Audacity、Adobe Audition等) 检查录音和回采**通道是否正常获取到了音频数据，不要通过直接用命令播放录到的wav文件或者直接在板级上用播放器来验证**

下面将逐一介绍上述回采方案

2.1、硬回采+单个CPU DAI

以rk809播放、es7210 2mic录音+2ref回采为例，其拓扑关系图如下所示



如上图所示采用的是硬回采（有实际的loopback回采电路），录音和回采都通过es7210经由单个硬件协议（i2s或者pdm）传入soc，这种情况直接按录音和播放声卡的正常配置来处理即可无需特殊处理

dts配置

其中录音声卡按simple-card或者multicodec方式正常配置均可，参考配置如下，其中详细属性可参考[Rockchip Developer Guide Audio CN V2.4.0.pdf](#)，后续不再赘述

```

/* es7210参考配置，按正常配置即可无需额外修改 */
es7210_sound: es7210-sound {
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip-es7210";
    rockchip,format = "i2s";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&i2s0_8ch>;
    rockchip,codec = <&es7210>;
    dai-tdm-slot-num = <4>;//slot的数量，这里以tdm4为例，tdm模式下才需要配置
    dai-tdm-slot-width = <32>;//slot的宽度，tdm模式下才需要配置
    status = "okay";
};

&i2s0_8ch {
    status = "okay";
    pinctrl-0 = <&i2s0_lrck
        &i2s0_sclk
        &i2s0_sdi0
        &i2s0_sdo0>;
};

es7210: es7210@40 {
    status = "okay";
    #sound-dai-cells = <0>;
    compatible = "MicArray_0";
    reg = <0x40>;
    clocks = <&mclkout_i2s0>;
    clock-names = "mclk";
    assigned-clocks = <&mclkout_i2s0>;
    assigned-clock-rates = <12288000>;
};

/* rk809参考配置，此处只做播放声卡考虑 */
rk809_sound: rk809-sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,format = "i2s";
    simple-audio-card,name = "rockchip-rk809";

```

```

simple-audio-card,mclk-fs = <256>;
simple-audio-card,widgets =
    "Microphone", "Mic Jack",
    "Headphone", "Headphone Jack";
simple-audio-card,routing =
    "Mic Jack", "MICBIAS1",
    "IN1P", "Mic Jack",
    "Headphone Jack", "HPOL",
    "Headphone Jack", "HPOR";
simple-audio-card,cpu {
    sound-dai = <&i2s1_8ch>;
};
simple-audio-card,codec {
    sound-dai = <&rk809_codec>;
};
};

<u>&i2s1_8ch {</u>
    status = "okay";
    rockchip,clk-trcm = <1>;//i2s接口推荐加上该属性，如果是6.1即后续版本的内核则使用
    rockchip,trcm-sync-tx-only;
    pinctrl-names = "default";
    pinctrl-0 = <&i2s1m0_sclk
        &i2s1m0_lrck
        &i2s1m0_sdi0
        &i2s1m0_sdo0>;
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mclkout_i2s1>;
    clock-names = "mclk";
    assigned-clocks = <&mclkout_i2s1>;
    assigned-clock-rates = <12288000>;
    use-ext-amplifier;//使用外部功放时才使用
    hp-volume = <20>;
    spk-volume = <3>;
    status = "okay";
};

```

验证通路

声卡正常生成后需要验证录音+回采是否生效，**要基于基本音频命令来验证通路**，Linux平台通过arecord/aplay来验证，Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成，cat proc/asound/cards查看是否生成了对应播放和录音的两张声卡（对于ipc类sdk由于不带alsa库不能用这类方法）

```

root@rk3588-buildroot:/# cat proc/asound/cards
0 [rockchipes7210 ]: rockchip-es7210 - rockchip-es7210
                      rockchip-es7210
1 [rockchiprk809  ]: rockchip-rk809 - rockchip-rk809
                      rockchip-rk809

```

2. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），**如果基本的播放和录音都有问题请先检查amixer控件信息**（安卓为tinymix）

```

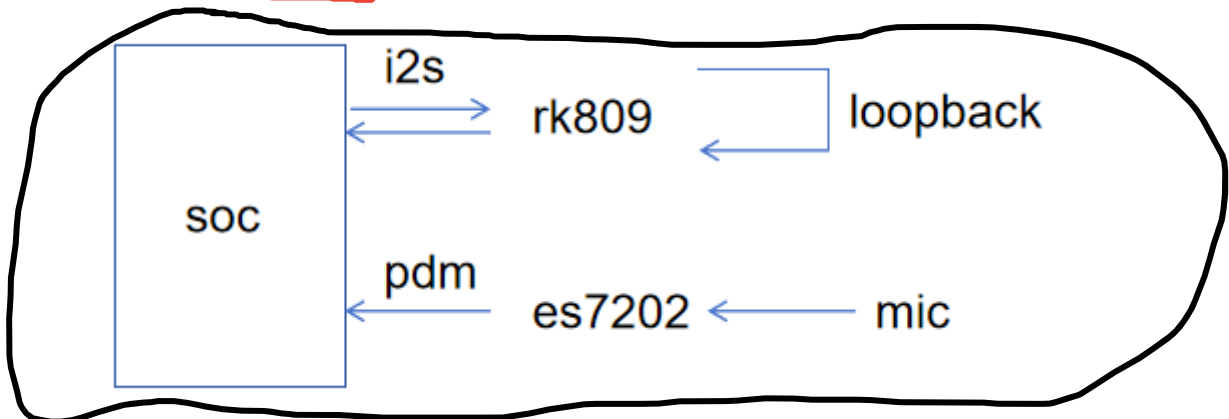
root@rk3588-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav & //将aplay播放放到后台执行，且aplay播放wav不用指定格式
root@rk3588-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4 /tmp/arecord.wav //aplay播放出声的同时arecord录2mic+2ref，声道数需和配置保持一致
root@rk3588-buildroot:/# amixer -c 0 contents //该步骤为异常情况下的检查示例，前面步骤都正常的话可无需执行

```

3. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

2.2、硬回采+多个CPU DAI

以rk809播放，rk809 2ref回采+es7202 2mic录音为例，其拓扑关系图如下所示



如上图所示采用的是硬回采，录音和回采分别通过es7202和rk809经由不同的硬件协议（pdm和i2s）传入soc，这种情况涉及到多cpu dai的情况因此需要采用multi-dai的配置

内核配置

```
CONFIG_SND_SOC_ROCKCHIP_MULTI_DAI //multi-dai编译选项
```

dts配置

参考配置及其解释如下：

```

multi_sound:multi-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip-multi";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&multi_dais>;
    rockchip,codec = <&es7202>, <&rk809>;
};

multi_dais: multi-dais {
    status = "okay";
    compatible = "rockchip,multi-dais";
    dais = <&pdm1> <&i2s1_8ch>;
    capture,channel-mapping = <2 2>; //实际所需声道数，与dais属性顺序对应
    playback,channel-mapping = <0 2>; //同上
    #sound-dai-cells = <0>;
};

&pdm1 {

```

```

status = "okay";
sound-name-prefix = "PDM1"; //使用multi-dai时最好标注节点名，尤其涉及到例如两个不同
i2s或sai的情况
rockchip,no-dmaengine; //使用multi-dai时必配属性，正常使用不需要
rockchip,path-map = <3 2 1 0>; //pdm通道映射功能，这里用以将sdi3、sdi2放在前4ch，根
据实际情况调整
pinctrl-names = "default";
pinctrl-0 = <&pdm1m0_clk0
    &pdm1m0_clk1
    &pdm1m0_sdi2
    &pdm1m0_sdi3>;
};

&i2s1_8ch {
status = "okay";
sound-name-prefix = "I2S1";
rockchip,no-dmaengine;
rockchip,clk-trcm = <1>; //i2s接口推荐加上该属性，如果是6.1即后续版本的内核则使用
rockchip,trcm-sync-tx-only;
pinctrl-names = "default";
pinctrl-0 = <&i2s1m0_sclk
    &i2s1m0_lrck
    &i2s1m0_sdi0
    &i2s1m0_sdo0>;
};

es7202: es7202@32 {
status = "okay";
#sound-dai-cells = <0>;
compatible = "ES7202_PDM_ADC_1";
power-supply = <&avcc_1v8_s0>; /* only 1v8 or 3v3, default is 3v3 */
reg = <0x32>;
};

rk809_codec: codec {
#sound-dai-cells = <1>;
compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
clocks = <&mclkout_i2s1>;
clock-names = "mclk";
assigned-clocks = <&mclkout_i2s1>;
assigned-clock-rates = <12288000>;
adc-for-loopback; //经由rk809的硬回采必须配置该属性
hp-volume = <20>;
spk-volume = <3>;
status = "okay";
};

```

注意事项:

1. 通常情况下，是否使用multi-dais主要关注录音和回采是否只通过一个cpu dai传输，如果不是就需要用到multi-dai，涉及到特殊场景请联系rk咨询处理
2. channel-mapping的配置需要和dais中的顺序对应，如上文的配法对应pdm1录2ch，i2s1_8ch录2ch播放2ch
3. 实际录音和播放通道必须与channel-mapping的数量相符，比如capture,channel-mapping = <2 2>，那就必须按照2+2=4ch来录音，同理必须按照2ch播放
4. 由于驱动的限制，multi-dai能混合的cpu dai的数量最多不能超过4个，比如示例中pdm1+i2s1就是2个符合要求

验证通路

声卡正常生成后需要验证录音+回采是否生效，要基于基本音频命令来验证通路，Linux平台通过arecord/aplay来验证，Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成，cat proc/asound/cards查看是否生成了multi-dai的对应声卡（对于ipc类sdk由于不带alsa库不能用这类方法）

```
root@rk3588-buildroot:/# cat proc/asound/cards
0 [rockchipmulti]: rockchip-multi - rockchip-multi
rockchip-multi
```

2. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），如果基本的播放和录音都有问题请先检查amixer控件信息（安卓为tinymix）

```
root@rk3588-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav & //将aplay播放放到后台执行，且aplay播放wav不用指定格式
root@rk3588-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4 /tmp/arecord.wav //aplay播放出声的同时arecord录2mic+2ref，声道数需和capture, channel-mapping中的配置保持一致
root@rk3588-buildroot:/# amixer -c 0 contents //该步骤为异常情况下的检查示例，前面步骤都正常的话可无需执行
```

3. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

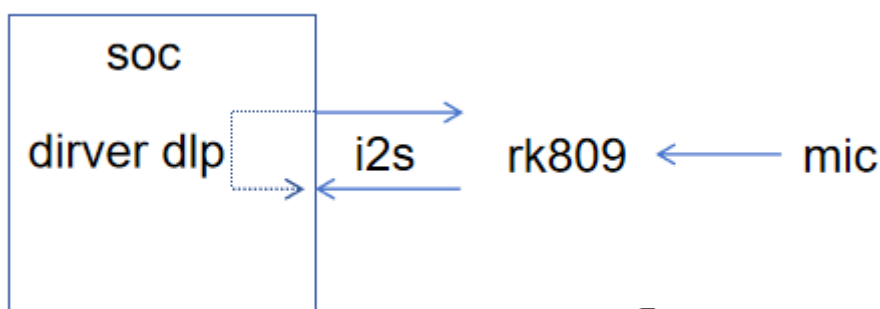
可以看到与 2.1小节硬回采+单个CPU DAI方案相比差距主要体现在dto配置上，这里把播放、录音与回采集成到了同一张声卡中，而验证通路的方式没有太大变化

2.3、软回采+单个CPU DAI

以rk809播放，rk809 2ref软回采+2mic录音为例，此时rk809没有外接loopback电路

2.3.1、通用方案 (DLP)

对于不带sai的平台（如rk356x, rk3588等）采用dlp的通用方案



该方案没有外围回采电路，回采数据通过音频驱动的dlp框架，经由I2S的2路播放数据复制准备后得到，再与2路mic->rk809->i2s拿到的录音数据组合起来传入soc

内核配置

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM //dlp软回采编译选项
```

dts配置

参考配置及其解释如下:

```
rk809_sound: rk809-sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,format = "i2s";
    simple-audio-card,name = "rockchip-rk809";
    simple-audio-card,mclk-fs = <256>;
    simple-audio-card,widgets =
        "Microphone", "Mic Jack",
        "Headphone", "Headphone Jack";
    simple-audio-card,routing =
        "Mic Jack", "MICBIAS1",
        "IN1P", "Mic Jack",
        "Headphone Jack", "HPOL",
        "Headphone Jack", "HPOR";
    simple-audio-card,cpu {
        sound-dai = <&i2s1_8ch>;
    };
    simple-audio-card,codec {
        sound-dai = <&rk809_codec>;
    };
};
```

dai → SAI / i2s / PDM →

```
&i2s1_8ch {
    status = "okay";
    rockchip,clk-trcm = <1>;//i2s接口推荐加上该属性，如果是6.1即后续版本的内核则使用
    rockchip,trcm-sync-tx-only;
    rockchip,digital-loopback;//Driver DLP需要在回采对应的cpu dai节点中增加这个属性
    pinctrl-names = "default";
    pinctrl-0 = <&i2s1m0_sclk
        &i2s1m0_lrck
        &i2s1m0_sdi0
        &i2s1m0_sdo0>;
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mclkout_i2s1>;
    clock-names = "mclk";
    assigned-clocks = <&mclkout_i2s1>;
    assigned-clock-rates = <12288000>;
    use-ext-amplifier;//使用外部功放时才使用
    hp-volume = <20>;
    spk-volume = <3>;
    status = "okay";
};
```

注意事项:

1. 参考2.2小节中dts配置的注意事项
2. 与硬回采相比，driver dlp软回采多了注册dlp属性的动作，dts配置中体现为需要在软回采对应的cpu dai节点中增加rockchip,digital-loopback属性

验证通路

声卡正常生成后需要验证录音+回采是否生效，**要基于基本音频命令来验证通路**，Linux平台通过arecord/aplay来验证，Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成，cat proc/asound/cards查看是否生成了multi-dai的对应声卡（对于ipc类sdk由于不带alsa库不能用这类方法）

```
root@rk3588-buildroot:/# cat proc/asound/cards
0 [rockchiprk809 ]: rockchip-rk809 - rockchip-rk809
rockchip-rk809
```

2. 通过amixer设置软回采必要的控件信息，打开dlp通路（安卓为tinymix）

```
root@rk3588-buildroot:/# amixer -c 0 contents
...
numid=10,iface=MIXER,name='Software Digital Loopback Mode'//找到这个控件
; type=ENUMERATED,access=rw-----,values=1,items=17
; Item #0 'Disabled'
; Item #1 '2CH: 1 Loopback + 1 Mic'
; Item #2 '2CH: 1 Mic + 1 Loopback'
; Item #3 '2CH: 1 Mic + 1 Loopback-mixed'
; Item #4 '2CH: 2 Loopbacks'
; Item #5 '4CH: 2 Mics + 2 Loopbacks'
; Item #6 '4CH: 2 Mics + 1 Loopback-mixed'
; Item #7 '4CH: 4 Loopbacks'
; Item #8 '6CH: 4 Mics + 2 Loopbacks'
; Item #9 '6CH: 4 Mics + 1 Loopback-mixed'
; Item #10 '6CH: 6 Loopbacks'
; Item #11 '8CH: 6 Mics + 2 Loopbacks'
; Item #12 '8CH: 6 Mics + 1 Loopback-mixed'
; Item #13 '8CH: 8 Loopbacks'
; Item #14 '10CH: 8 Mics + 2 Loopbacks'
; Item #15 '10CH: 8 Mics + 1 Loopback-mixed'
; Item #16 '16CH: 8 Mics + 8 Loopbacks'
: values=0
...
root@rk3588-buildroot:/# amixer -c 0 cset name='Software Digital Loopback
Mode' 5//根据实际情况选择，这里对应的选择是'4CH: 2 Mics + 2 Loopbacks'
```

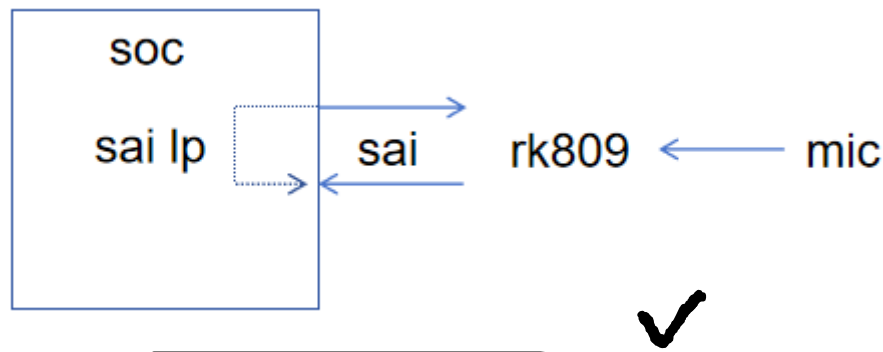
3. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），**如果基本的播放和录音都有问题请先检查amixer控件信息**

```
root@rk3588-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav &//将aplay播
放放到后台执行，且aplay播放wav不用指定格式
root@rk3588-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4
/tmp/arecord.wav//aplay播放出声的同时arecord录2mic+2ref，声道数需和
capture,channel-mapping中的配置保持一致，multi-dai声卡录音的采样率需要与播放wav文件的
采样率保持一致
root@rk3588-buildroot:/# amixer -c 0 contents//该步骤为异常情况下的检查示例，前面
步骤都正常的话可无需执行
```

4. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

2.3.2、SAI LP方案

对于带sai接口的平台（如rk3562、rk3576等）可以采用SAI LP的方案



该方案没有外围回采电路，回采数据通过通过配置芯片内部SAI控制器loopback开关获取，无需外部物理连线，再与2路mic->rk809->i2s拿到的录音数据组合起来传入soc

dts配置

参考配置及其解释如下：

```
rk809_sound: rk809-sound {
    status = "okay";
    compatible = "simple-audio-card";
    simple-audio-card,format = "i2s";
    simple-audio-card,name = "rockchip-rk809";
    simple-audio-card,mclk-fs = <256>;
    simple-audio-card,widgets =
        "Microphone", "Mic Jack",
        "Headphone", "Headphone Jack";
    simple-audio-card,routing =
        "Mic Jack", "MICBIAS1",
        "IN1P", "Mic Jack",
        "Headphone Jack", "HPOL",
        "Headphone Jack", "HPOR";
    simple-audio-card,cpu {
        sound-dai = <&sai1>;
    };
    simple-audio-card,codec {
        sound-dai = <&rk809_codec>;
    };
};

&sai1 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&sai1m0_lrck
        &sai1m0_sclk
        &sai1m0_sdi0
        &sai1m0_sdo0>;
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mclkout_sai1>;
    clock-names = "mclk";
```

```

assigned-clocks = <&mclkout_sai1>;
assigned-clock-rates = <12288000>;
pinctrl-names = "default";
pinctrl-0 = <&sai1m0_mclk>;
adc-for-loopback;
hp-volume = <20>;
spk-volume = <3>;
status = "okay";
};

```

注意事项:

1. 参考2.2小节中dts配置的注意事项
2. 与d1p方案相比，回采通过芯片内部sai接口回环获取，dts无需d1p的配置

验证通路

声卡正常生成后需要验证录音+回采是否生效，**要基于基本音频命令来验证通路**，Linux平台通过arecord/aplay来验证，Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成，cat proc/asound/cards查看是否生成了multi-dai的对应声卡（对于ipc类sdk由于不带alsa库不能用这类方法）

```

root@rk3576-buildroot:/# cat proc/asound/cards
0 [rockchiprk809 ]: rockchip-rk809 - rockchip-rk809
rockchip-rk809

```

2. 通过amixer设置软回采必要的控件信息，打开sai lp通路（安卓为tinymix）

```

root@rk3576-buildroot:/# amixer -c 0 contents
...
numid=19,iface=MIXER,name='SAI1 SDI1 Loopback Src Select'//这里因为是4ch, sdi0
获取mic的录音数据所以以sai1 sdi1回采为例
; type=ENUMERATED,access=rw-----,values=1,items=4
; Item #0 'From SD00'
; Item #1 'From SD01'
; Item #2 'From SD02'
; Item #3 'From SD03'
: values=0

numid=23,iface=MIXER,name='SAI1 SDI1 Loopback Switch'//sai lp使能开关
; type=ENUMERATED,access=rw-----,values=1,items=2
; Item #0 'Disable'
; Item #1 'Enable'
: values=0
...
root@rk3576-buildroot:/# amixer -c 0 cset name='SAI1 SDI0 Loopback Switch'
1//使能回采

```

3. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），**如果基本的播放和录音都有问题请先检查amixer控件信息**

```

root@rk3576-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav & //将aplay播放放到后台执行，且aplay播放wav不用指定格式
root@rk3576-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4 /tmp/arecord.wav //aplay播放出声的同时arecord录2mic+2ref，声道数需和capture, channel-mapping中的配置保持一致，multi-dai声卡录音的采样率需要与播放wav文件的采样率保持一致
root@rk3576-buildroot:/# amixer -c 0 contents //该步骤为异常情况下的检查示例，前面步骤都正常的话可无需执行

```

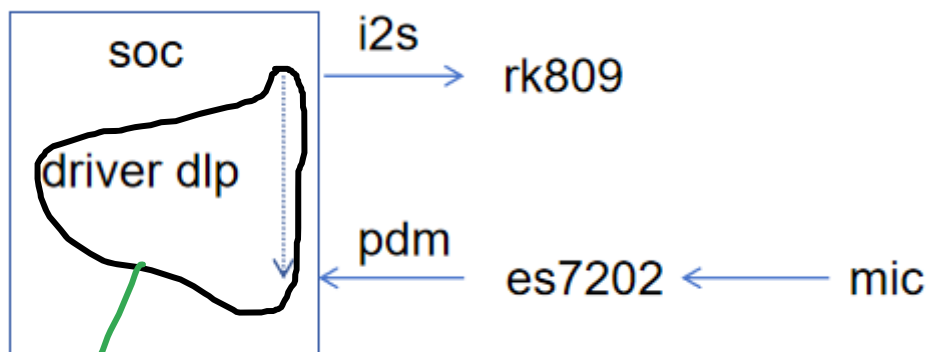
4. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

2.4、软回采+多个CPU DAI

以rk809播放，rk809 2ref软回采+es7202 2mic录音为例，此时rk809没有外接loopback电路

2.4.1、通用方案（DLP）

对于不带sai的平台（如rk356x，rk3588等）采用dlp的通用方案



该方案没有外围回采电路，回采数据通过音频驱动的dlp框架，经由I2S的2路播放数据复制准备后得到，再通过multi-dai驱动与2路mic->es7202->pdm拿到的录音数据组合起来传入soc

内核配置

```

CONFIG_SND_SOC_ROCKCHIP_DLP_PCM //dlp软回采编译选项
CONFIG_SND_SOC_ROCKCHIP_MULTI_DAI //multi-dai编译选项

```

dts配置

参考配置及其解释如下：

```

multi_sound:multi-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip,multi";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&multi_dais>;
    rockchip,codec = <&es7202>, <&rk809>;
};

multi_dais: multi-dais {
    status = "okay";
    compatible = "rockchip,multi-dais";
    dais = <&pdm1>, <&i2s1_8ch>;
};

```

```

capture,channel-mapping = <2 2>;//实际所需声道数，与dais属性顺序对应
playback,channel-mapping = <0 2>;//同上
#sound-dai-cells = <0>;
};

&pdm1 {
    status = "okay";
    sound-name-prefix = "PDM1";//使用multi-dai时最好标注节点名，尤其涉及到例如两个不同
i2s或sai的情况
    rockchip,no-dmaengine;//使用multi-dai时必配属性，正常使用不需要
    rockchip,path-map = <3 2 1 0>;//pdm通道映射功能，这里用以将sdi3、sdi2放在前4ch，根
据实际情况调整
    pinctrl-names = "default";
    pinctrl-0 = <&pdm1m0_clk0
        &pdm1m0_clk1
        &pdm1m0_sdi2
        &pdm1m0_sdi3>;
};

&i2s1_8ch {
    status = "okay";
    sound-name-prefix = "I2S1";
    rockchip,no-dmaengine;
    rockchip,digital-loopback;//Driver DLP需要在回采对应的cpu dai节点中增加这个属性
    rockchip,clk-trcm = <1>;//i2s接口推荐加上该属性，如果是6.1即后续版本的内核则使用
rockchip,trcm-sync-tx-only;
    pinctrl-names = "default";
    pinctrl-0 = <&i2s1m0_sclk
        &i2s1m0_lrck
        &i2s1m0_sdi0
        &i2s1m0_sdo0>;
};

es7202: es7202@32 {
    status = "okay";
    #sound-dai-cells = <0>;
    compatible = "ES7202_PDM_ADC_1";
    power-supply = <&avcc_1v8_s0>; /* only 1v8 or 3v3, default is 3v3 */
    reg = <0x32>;
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mclkout_i2s1>;
    clock-names = "mclk";
    assigned-clocks = <&mclkout_i2s1>;
    assigned-clock-rates = <12288000>;
    adc-for-loopback;
    hp-volume = <20>;
    spk-volume = <3>;
    status = "okay";
};

```

注意事项:

1. 参考2.2小节中dts配置的注意事项

2. 与硬回采相比，driver dlp软回采多了注册dlp属性的动作，dts配置中体现为需要在软回采对应的cpu dai节点中增加rockchip,digital-loopback属性

验证通路

声卡正常生成后需要验证录音+回采是否生效，**要基于基本音频命令来验证通路**，Linux平台通过arecord/aplay来验证，Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成，cat proc/asound/cards查看是否生成了multi-dai的对应声卡（对于ipc类sdk由于不带alsa库不能用这类方法）

```
root@rk3588-buildroot:/# cat proc/asound/cards
0 [rockchipmulti ]: rockchip-multi - rockchip-multi
                      rockchip-multi
```

2. 通过amixer设置软回采必要的控件信息，打开dlp通路（安卓为tinymix）

```
root@rk3588-buildroot:/# amixer -c 0 contents
...
numid=26,iface=MIXER,name='Software Digital Loopback Mode'//找到这个控件
; type=ENUMERATED,access=rw-----,values=1,items=17
; Item #0 'Disabled'
; Item #1 '2CH: 1 Loopback + 1 Mic'
; Item #2 '2CH: 1 Mic + 1 Loopback'
; Item #3 '2CH: 1 Mic + 1 Loopback-mixed'
; Item #4 '2CH: 2 Loopbacks'
; Item #5 '4CH: 2 Mics + 2 Loopbacks'
; Item #6 '4CH: 2 Mics + 1 Loopback-mixed'
; Item #7 '4CH: 4 Loopbacks'
; Item #8 '6CH: 4 Mics + 2 Loopbacks'
; Item #9 '6CH: 4 Mics + 1 Loopback-mixed'
; Item #10 '6CH: 6 Loopbacks'
; Item #11 '8CH: 6 Mics + 2 Loopbacks'
; Item #12 '8CH: 6 Mics + 1 Loopback-mixed'
; Item #13 '8CH: 8 Loopbacks'
; Item #14 '10CH: 8 Mics + 2 Loopbacks'
; Item #15 '10CH: 8 Mics + 1 Loopback-mixed'
; Item #16 '16CH: 8 Mics + 8 Loopbacks'
: values=0
...
root@rk3588-buildroot:/# amixer -c 0 cset name='Software Digital Loopback
Mode' 5//根据实际情况选择，这里对应的选择是'4CH: 2 Mics + 2 Loopbacks'
```

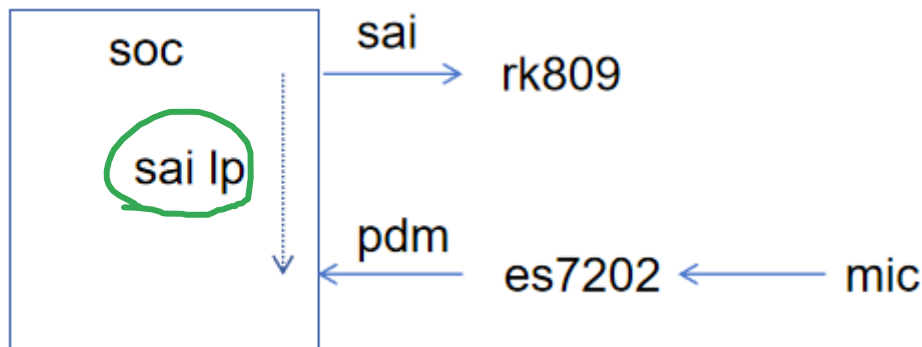
3. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），**如果基本的播放和录音都有问题请先检查amixer控件信息**

```
root@rk3588-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav &//将aplay播
放放到后台执行，且aplay播放wav不用指定格式
root@rk3588-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4
/tmp/arecord.wav//aplay播放出声的同时arecord录2mic+2ref，声道数需和
capture,channel-mapping中的配置保持一致，multi-dai声卡录音的采样率需要与播放wav文件的
采样率保持一致
root@rk3588-buildroot:/# amixer -c 0 contents//该步骤为异常情况下的检查示例，前面
步骤都正常的话可无需执行
```

4. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

2.4.2、SAI LP方案

对于带sai接口的平台（如rk3562、rk3576等）可以采用SAI LP的方案



该方案没有外围回采电路，回采数据通过通过配置芯片内部SAI控制器loopback开关获取，无需外部物理连线，再通过multi-dai驱动与2路mic->es7202->pdm拿到的录音数据组合起来传入soc

内核配置

```
CONFIG_SND_SOC_ROCKCHIP_MULTI_DAI5//multi-dai编译选项
```

dts配置

参考配置及其解释如下：

```
multi_sound:multi-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip,multi";
    rockchip,mclk-fs = <256>;
    rockchip,cpu = <&multi_dais>;
    rockchip,codec = <&es7202>, <&rk809>;
};

multi_dais: multi-dais {
    status = "okay";
    compatible = "rockchip,multi-dais";
    dais = <&pdm1>, <&sai1>;
    capture,channel-mapping = <2 2>;//实际所需声道数，与dais属性顺序对应
    playback,channel-mapping = <0 2>;//同上
    #sound-dai-cells = <0>;
};

&pdm1 {
    status = "okay";
    sound-name-prefix = "PDM1";//使用multi-dai时最好标注节点名，尤其涉及到例如两个不同
i2s或sai的情况
    rockchip,no-dmaengine;//使用multi-dai时必配属性，正常使用不需要
    rockchip,path-map = <3 2 1 0>;//pdm通道映射功能，这里用以将sdi3、sdi2放在前4ch，根据
实际情况调整
    pinctrl-names = "default";
    pinctrl-0 = <&pdm1m0_clk0
        &pdm1m0_clk1
```

```

        &pdm1m0_sdi2
        &pdm1m0_sdi3>;
};

&sai1 {
    status = "okay";
    sound-name-prefix = "SAI1";
    pinctrl-names = "default";
    pinctrl-0 = <&sai1m0_lrck
        &sai1m0_sclk
        &sai1m0_sdi0
        &sai1m0_sdo0>;
};

es7202: es7202@32 {
    status = "okay";
    #sound-dai-cells = <0>;
    compatible = "ES7202_PDM_ADC_1";
    power-supply = <&avcc_1v8_s0>; /* only 1v8 or 3v3, default is 3v3 */
    reg = <0x32>;
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mclkout_sai1>;
    clock-names = "mclk";
    assigned-clocks = <&mclkout_sai1>;
    assigned-clock-rates = <12288000>;
    pinctrl-names = "default";
    pinctrl-0 = <&sai1m0_mclk>;
    adc-for-loopback;
    hp-volume = <20>;
    spk-volume = <3>;
    status = "okay";
};

```

注意事项:

1. 参考2.2小节中dts配置的注意事项
2. 与d1p方案相比, 回采通过芯片内部sai接口回环获取, dts无需d1p的配置

验证通路

声卡正常生成后需要验证录音+回采是否生效, 要基于基本音频命令来验证通路, Linux平台通过arecord/aplay来验证, Android平台通过tinycap/tinyplay来验证

1. 确认声卡生成, cat proc/asound/cards查看是否生成了multi-dai的对应声卡 (对于ipc类sdk由于不带alsa库不能用这类方法)

```

root@rk3576-buildroot:/# cat proc/asound/cards
0 [rockchipmulti]: rockchip-multi - rockchip-multi
                    rockchip-multi

```

2. 通过amixer设置软回采必要的控件信息, 打开sai lp通路 (安卓为tinymix)

```

root@rk3576-buildroot:/# amixer -c 0 contents

```

```

...
numid=19,iface=MIXER,name='SAI1 SDI1 Loopback Src Select'//这里因为是4ch, sdi0
获取mic的录音数据所以以sai1 sdi1回采为例
; type=ENUMERATED,access=rw-----,values=1,items=4
; Item #0 'From SD00'
; Item #1 'From SD01'
; Item #2 'From SD02'
; Item #3 'From SD03'
: values=0

numid=23,iface=MIXER,name='SAI1 SDI1 Loopback Switch'//sai 1p使能开关
; type=ENUMERATED,access=rw-----,values=1,items=2
; Item #0 'Disable'
; Item #1 'Enable'
: values=0
...
root@rk3576-buildroot:/# amixer -c 0 cset name='SAI1 SDI0 Loopback Switch'
1//使能回采

```

3. aplay播放1khz正弦波验证播放通路是否打通，确认后arecord录4ch wav文件（tinycap/tinyplay同理），**如果基本的播放和录音都有问题请先检查amixer控件信息**

```

root@rk3576-buildroot:/# aplay -D hw:0,0 /tmp/48k_2ch_16bit.wav &//将aplay播
放放到后台执行，且aplay播放wav不用指定格式
root@rk3576-buildroot:/# arecord -D hw:0,0 -f S16_LE -r 48000 -c 4
/tmp/arecord.wav//aplay播放出声的同时arecord录2mic+2ref，声道数需和
capture,channel-mapping中的配置保持一致，multi-dai声卡录音的采样率需要与播放wav文件的
采样率保持一致
root@rk3576-buildroot:/# amixer -c 0 contents//该步骤为异常情况下的检查示例，前面
步骤都正常的话可无需执行

```

4. 将录到的音频文件adb pull出来，通过音频分析软件（如Audacity、Adobe Audition等）查看是否正常录到了麦克风和回采的音频数据

3、特殊方案介绍

以上主要针对大部分常见场景，有些回采场景可能涉及到特殊配置，下面将进行针对介绍

3.1、HDMI 回采

受限于hdmI协议特性，与spk回采相比**hdmI回采无法通过外围loopback电路完成**，在回采场景中需要将其理解成一个特殊的输出设备，这类场景通常会在安卓box方案中遇到，因此本节主要以安卓平台为例进行介绍

首先介绍**安卓平台的音频特性**：

1. 录音和播放是分开的通路，两者在软件层面上(hal层及以上)互不影响使用的是不同的strategy
2. 对于播放（录音）通路，安卓平台默认只能同时打开一张声卡，也就是说如果涉及到两个或以上播放（录音）声卡同时使用的情况安卓默认不支持，对于安卓平台而言默认一个音频strategy对应一个route、一个stream、一个device和一个card，strategy会根据策略优先级来分配，例如hdmI的策略优先级默认高于spk，使用了hdmI策略就不再使用spk策略
3. 无论是hdmI回采还是spk回采对于安卓来说同属于BUILTIN_MIC的strategy和device，两者对于上层来说只存在声卡名的区别，**因此如果配置了spk回采的声卡，安卓再无法再识别到hdmI回采的声卡，这里存在回采声卡竞争的问题**

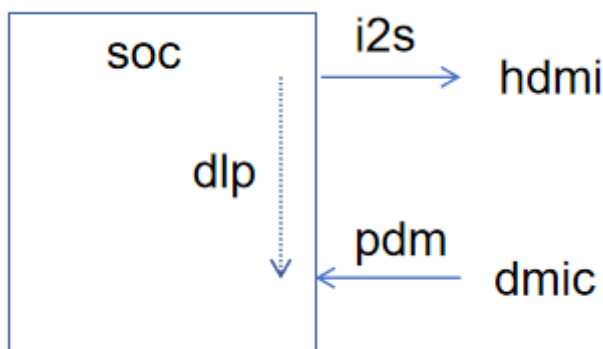
基于以上分析可以得到以下两个关键问题：

1. 当音频输出（输入）设备的数量和具体方案不同时，会不会出现同一通路下的竞态问题，例如当hdmi和spk都需要回采时该如何处理这里的声卡竞争
2. 由于hdmi回采无法直接通过外围loopback电路完成，那么怎样获取到hdmi的回采数据

这两个问题决定了hdmi回采方案的软硬件选择，下面将就此对常见的hdmi回采方案进行介绍（以下均基于Android 14部署）

3.1.1、单HDMI输出

这类场景中**输出设备只有hdmi**，因此不存在竞态问题，将hdmi当作特殊的输出设备处理即可，通过hdmi的软回采来获取回采数据



内核配置

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM//d1p软回采编译选项
CONFIG_SND_SOC_ROCKCHIP_MULTI_DAI5//multi-dai编译选项
```

dts配置

```
/*这里以通解d1p方案为例，如果是3576等带sai 1p的平台请参考2.4.2小节中的配置*/
multi_sound: multi-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip-multi";
    rockchip,format = "i2s";
    rockchip,mclk-fs = <128>;
    rockchip,cpu = <&multi_dais>;
    rockchip,codec = <&dummy_codec &hdmi0>;
};

multi_dais: multi-dais {
    status = "okay";
    compatible = "rockchip,multi-dais";
    dais = <&pdm0>, <&i2s5_8ch>;
    capture,channel-mapping = <4 2>;
    playback,channel-mapping = <0 2>;
    #sound-dai-cells = <0>;
    rockchip,jack-det;
};

&pdm0 {
    sound-name-prefix = "PDM0";
    rockchip,no-dmaengine;
    status = "okay";
};
```

```

&i2s5_8ch {
    sound-name-prefix = "I2S5";
    rockchip,no-dmaengine;
    rockchip,digital-loopback;
    status = "okay";
};

dummy_codec: dummy-codec {
    compatible = "rockchip,dummy-codec";
    #sound-dai-cells = <0>;
    status = "okay";
};

&hdmi0 { //这部分直接用sdk里默认的通常不用改，这里只做参考
    enable-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    pinctrl-0 = <&hdmi0_tx0_cec &hdmi0_tx0_hpd &hdmi0_tx0_scl
&hdmi0_tx0_sda>;
    status = "okay";
};

```

驱动修改

hdmi的codec驱动默认不支持同时录音播放，参考以下打上补丁

```

~/workspace/rk_android14/kernel-6.1$ git diff sound/soc/codecs/hdmi-codec.c
diff --git a/sound/soc/codecs/hdmi-codec.c b/sound/soc/codecs/hdmi-codec.c
index 6a51f83e246d..568f74ddc114 100644
--- a/sound/soc/codecs/hdmi-codec.c
+++ b/sound/soc/codecs/hdmi-codec.c
@@ -479,11 +479,11 @@ static int hdmi_codec_startup(struct snd_pcm_substream
*substream,
        return 0;

        mutex_lock(&hcp->lock);
-       if (hcp->busy) {
+       /*if (hcp->busy) {
            dev_err(dai->dev, "Only one simultaneous stream supported!\n");
            mutex_unlock(&hcp->lock);
            return -EINVAL;
        }
+       }*/

        if (hcp->hcd.ops->audio_startup) {
            ret = hcp->hcd.ops->audio_startup(dai->dev->parent, hcp->hcd.data);

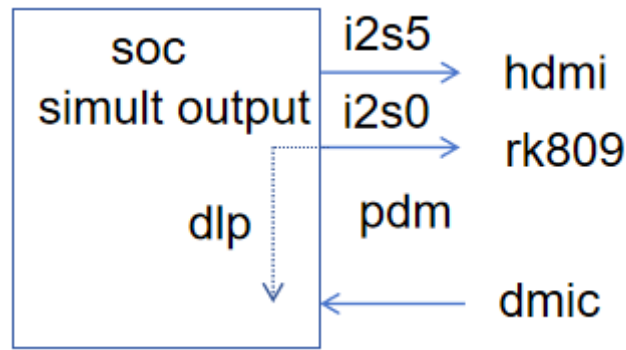
```

验证通路

参考2.4.1小节中软回采的验证方法（带sai接口的平台参考2.4.2小节中sai lp的做法）

3.1.2、SPK+HDMI同时输出

这类场景会涉及到喇叭和hdmi两个输出设备，两个设备同时播放同一个音源且都需要回采，由于播放的音源相同因此两个设备的输出波形理论上是近似的，**可以考虑把spk的回采当作hdmi的回采来处理**，这里通过转换的思路避免了hdmi回采和spk回采的声卡竞争问题



内核配置

CONFIG_SND_SOC_ROCKCHIP_DLP_PCM//d1p软回采编译选项
CONFIG_SND_SOC_ROCKCHIP_MULTI_DAI5//multi-dai编译选项

dts配置

```

/*这里以通解d1p方案为例，如果是3576等带sai 1p的平台请参考2.4.2小节中的配置*/
multi_sound: multi-sound {
    status = "okay";
    compatible = "rockchip,multicodecs-card";
    rockchip,card-name = "rockchip-multi";
    rockchip,format = "i2s";
    rockchip,mclk-fs = <128>;
    rockchip,cpu = <&multi_dais>;
    rockchip,codec = <&rk809_codec &hdmi0>;
};

multi_dais: multi-dais {
    status = "okay";
    compatible = "rockchip,multi-dais";
    dais = <&pdm0>, <&i2s1_8ch>;
    capture,channel-mapping = <4 2>;
    playback,channel-mapping = <0 2>;
    #sound-dai-cells = <0>;
    rockchip,jack-det;
};

&pdm0 {
    sound-name-prefix = "PDM0";
    rockchip,no-dmaengine;
    status = "okay";
};

&i2s1_8ch {
    status = "okay";
    rockchip,clk-trcm = <1>;//i2s接口推荐加上该属性，如果是6.1即后续版本的内核则使用
    rockchip,trcm-sync-tx-only;
    rockchip,digital-loopback;//Driver DLP需要在回采对应的cpu dai节点中增加这个属性
    pinctrl-names = "default";
    pinctrl-0 = <&i2s1m0_sclk
        &i2s1m0_lrck
        &i2s1m0_sdi0
        &i2s1m0_sdo0>;
};

```

```

dummy_codec: dummy-codec {
    compatible = "rockchip,dummy-codec";
    #sound-dai-cells = <0>;
    status = "okay";
};

rk809_codec: codec {
    #sound-dai-cells = <1>;
    compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
    clocks = <&mc1kout_i2s1>;
    clock-names = "mclk";
    assigned-clocks = <&mc1kout_i2s1>;
    assigned-clock-rates = <12288000>;
    adc-for-loopback;
    hp-volume = <20>;
    spk-volume = <3>;
    status = "okay";
};

```

hal层适配

这类方案需要hal层在打开播放声卡的时候同时打开hdmi和spk的声卡以达到同源输出的目的，请联系rk获取相关补丁

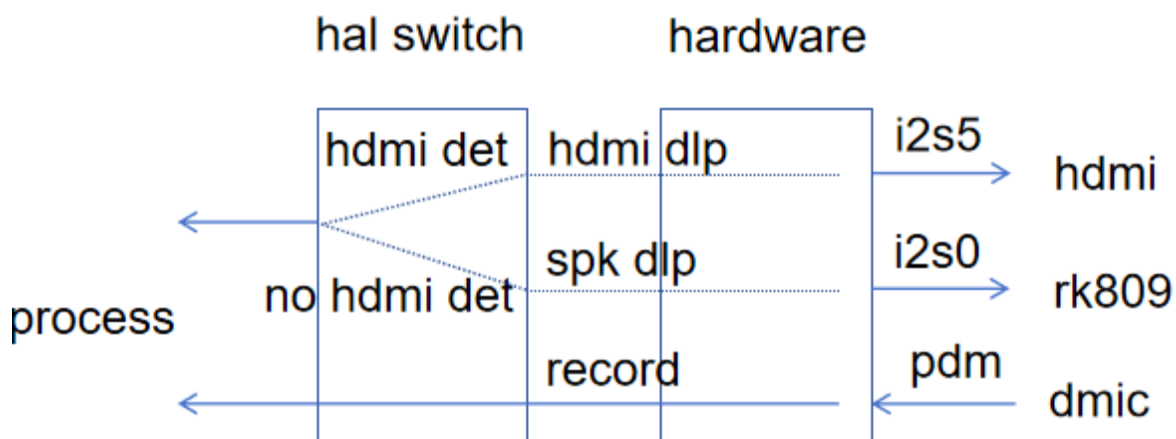
验证通路

参考2.4.1小节中软回采的验证方法（带sai接口的平台参考2.4.2小节中sai lp的做法）

3.1.3、SPK+HDMI独立输出

这类场景涉及到喇叭+hdmi两个输出设备，两个设备不能同时输出且都需要回采；这类场景最为复杂，需要根据客户实际的硬件拓扑关系和音频方案来考虑如何处理回采声卡的竞争问题，这里总的思路是，要区分hdmi和spk的回采数据要么把两路数据都拿到hal层然后在hal层做筛选，要么就在不同场景下共用同一个通路的回采数据，这里将介绍这两种方法：

1. 将mic录音+spk回采+hdmi回采配到一张声卡上，同时录取三个通路的音频数据来避免声卡竞争问题。客户若采用此方案需要自行在hal层进行适配，在录音通路中通过读取hdmi的状态来判断当前的实际状态（到底是hdmi输出还是spk输出），如果是hdmi输出则在算法前将丢弃spk回采数据只处理mic录音+hdmi回采，反之亦然；这个方案相当于在hal层实现一个switch功能，根据输出设备的选择来判断送给算法哪一路回采



2. 将mic录音+spk回采配到一张声卡上，把hdmi的回采想办法转换成spk回采来处理避免录音声卡的竞争。即当插入hdmi设备识别到hdmi时，走hdmi的策略，此时hal层同时打开spk和hdmi声卡并手动拉低spk的功放使能避免hdmi输出的同时spk同时出声，且不影响仍然通过spk的软回采通路来获取回采数据；而当拔出hdmi设备没有识别到hdmi时保持走原先spk的策略，这样就能在避免录音声卡竞态的前提下实现hdmi+spk独立输出与回采的方案

如果采用该方案，需要在codec驱动中透出使能gpio的节点，把gpio的控制放到hal层中来做，并且修改hal层中的控制逻辑，以下是参考：

```
/* 驱动修改 */
diff --git a/sound/soc/codecs/es8323.c b/sound/soc/codecs/es8323.c
index fc0b4a6a0281..1df8b7c14a2b 100644
--- a/sound/soc/codecs/es8323.c
+++ b/sound/soc/codecs/es8323.c
@@ -25,6 +25,7 @@
#include <sound/soc-dapm.h>
#include <sound/initval.h>
#include <linux/proc_fs.h>
+#include <linux/gpio/consumer.h>
#include "es8323.h"

#define NR_SUPPORTED_MCLK_LRCK_RATIOS 5
@@ -87,12 +88,13 @@ static struct reg_default es8323_reg_defaults[] = {

/* codec private data */
struct es8323_priv {
    struct regmap *regmap;
+    struct gpio_desc *gpiod; // 用于存储 SPK GPIO
};

static int es8323_reset(struct snd_soc_component *component)
@@ -749,6 +751,30 @@ static int es8323_probe(struct snd_soc_component
*component)
    struct es8323_priv *es8323 =
snd_soc_component_get_drvdata(component);
    int ret = 0;

+    // --- ADD THESE LINES ---
+    // 获取 GPIO
+    es8323->gpiod = devm_gpiod_get_optional(component->dev, "spk-con",
GPIO_OUT_LOW);
+    if (IS_ERR(es8323->gpiod)) {
+        dev_err(component->dev, "Failed to get spk-con-gpio\n");
+        return PTR_ERR(es8323->gpiod);
+    }
+
+    // 将 GPIO 导出到 /sys/class/gpio/, 允许应用层控制
+    // 第二个参数 false 表示初始方向为输出 (Output), 且不允许改变方向
+    if (es8323->gpiod) {
+        int ret = gpiod_export(es8323->gpiod, false);
+        if (ret) {
+            dev_err(component->dev, "Failed to export spk-con-gpio: %d\n",
ret);
+        }
+    }
+
+    // 在 /sys/class/sound/ 下创建软链接, 方便查找
```

```

+         ret = gpiod_export_link(component->dev, "spk-con-gpio", es8323-
>gpiod);
+         if (ret) {
+             dev_err(component->dev, "Failed to link spk-con-gpio: %d\n",
ret);
+         }
+     }
+     // --- END ADD ---
+
+     es8323->mclk = devm_clk_get(component->dev, "mclk");
+     if (IS_ERR(es8323->mclk)) {
+         dev_err(component->dev, "%s mclk is missing or invalid\n",
__func__);
+
+     }
+
+     /* hal层修改 */
+     /*
+      * @brief 通过 sysfs 接口设置指定 GPIO 的电平值
+      * @param value 要写入的值 (0 或 1)
+      */
+     static void set_spk_gpio(int value) {
+         const char *gpio_path = "/sys/class/gpio/gpioxx/value"; // 请将 "xx" 替换
为实际的 GPIO 编号
+         char buf[2];
+         int len;
+         int fd = open(gpio_path, O_WRONLY);
+         if (fd < 0) {
+             ALOGE("Failed to open SPK GPIO for write: %s", strerror(errno));
+             return;
+         }
+
+         len = snprintf(buf, sizeof(buf), "%d", value);
+         if (write(fd, buf, len) < 0) {
+             ALOGE("Failed to write SPK GPIO: %s", strerror(errno));
+         }
+         close(fd);
+     }
+
+     // --- 主逻辑函数 ---
+
+     /*
+      * @brief 根据音频输出设备类型，设置相应的路由策略和 GPIO 状态
+      * @param out 音频输出流的描述符指针
+      */
+     static void open_sound_card_policy(struct stream_out *out) {
+         if (out == NULL) {
+             return;
+         }
+
+         // 如果是比特流、多PCM或所有SCO设备，则跳过此策略
+         if (is_bitstream(out) || is_multi_pcm(out) || (out->device &
AUDIO_DEVICE_OUT_ALL_SCO)) {
+             return;
+         }
+
+         // 逻辑判断：根据连接的设备类型选择输出路由
+         // 注意：这里的 is_headset_connect() 和 is_hdmi_connected() 是假设存在的函数
+         // 你需要用你项目中实际的函数来替换它们。
+         if (is_headset_connect()) {

```

```

        out->device = AUDIO_DEVICE_OUT_WIRED_HEADSET;
        set_spk_gpio(0); // 插入耳机，关闭扬声器功放
        ALOGE("Use the headphone route");
    } else if (is_hdmi_connected()) {
        out->device = AUDIO_DEVICE_OUT_AUX_DIGITAL |
        AUDIO_DEVICE_OUT_SPEAKER;
        set_spk_gpio(0); // HDMI输出，关闭扬声器功放
        ALOGE("Use the HDMI route");
    } else {
        // 默认情况：无耳机无HDMI，使用扬声器
        out->device = AUDIO_DEVICE_OUT_SPEAKER;
        set_spk_gpio(1); // 使用扬声器，打开扬声器功放
        ALOGE("Use the SPK route");
    }
}

```

4、FAQ

4.1、multi声卡没有生成

具体表现可能为开机后cat proc/asound/cards没有找到对应声卡，开机log中发现Failed to register card: -517的加载报错

可能原因：

1. cpu dai节点中没有配置rockchip,no-dmaengine;的属性
2. multi-dai、dai对应的内核选项没有打开（SND_SOC_ROCKCHIP_MULTI_DAI、SND_SOC_ROCKCHIP_DAI等），这个根据实际情况编译完后在kernel-6.1(或其他内核版本)/sound/soc/rockchip目录下确认是否有正确生成对应的.o文件
3. 使用multi-sound节点时，原来的sound节点有没有关掉，注意不要出现节点复用的情况
4. cpu_dai是否出现了重复识别的情况，节点里是否有配置sound-name-prefix的属性标识
5. channel-mapping必须是双数倍的，单麦也要按照2ch来处理，不支持例如<1 1>的配置
6. 驱动版本老旧还不支持multi-dai功能，这种情况请联系rk更新音频驱动

4.2、使用arecord (tinycap) 录音报错

具体可表现可能为arecord: set_params:1398: Channels count non available或者hw_params等设置参数的报错

可能原因：

1. 录音的通道数与capture,channel-mapping的不一致（播放同理），使用multi-dai后这里必须按照规定的声道数量录音/播放，例如配置的是capture,channel-mapping = <2 2>那就必须是4ch录音
2. 驱动或者硬件不支持相关音频参数，对于安卓平台来说，可以tinypcm_info -D xx（声卡id）来查看播放和录音所支持的音频参数，根据所限制的参数来；如果是codec驱动的话声道数低于capture,channel-mapping所需的声道总和（比如驱动只支持2ch但是实际需要2+2=4ch），那需要修改驱动相关的部分，以rk809为例，这里对应的就是playback（播放）和capture（录音）的.channels_max要大于等于capture,channel-mapping的总和，其他音频驱动同理

```

static struct snd_soc_dai_driver rk817_dai[] = {
    {
        .name = "rk817-hifi",
        .id = RK817_HIFI,
        .playback = {
            .stream_name = "HiFi Playback",
            .channels_min = 2,

```

```

        .channels_max = 8, //关注这里
        .rates = RK817_PLAYBACK_RATES,
        .formats = RK817_FORMATS,
    },
    .capture = {
        .stream_name = "HiFi Capture",
        .channels_min = 2,
        .channels_max = 8, //关注这里
        .rates = RK817_CAPTURE_RATES,
        .formats = RK817_FORMATS,
    },
    .ops = &rk817_dai_ops,
},
...

```

4.3、先录音后播放回采声道数据丢失，顺序反过来就正常

具体表现可能为先arecord，然后再同时播放，录到的音频文件pull出来会发现录音通道正常而回采通道无音频数据，顺序反过来就正常

可能原因：

1. 同源问题。注意录音和播放的采样率是否相同，具体可在录音/播放时通过 `cat /proc/asound/card0/pcm0p(pcm0c)/sub0/hw_params` 确认。同源问题是指录音、回采和播放同时使用同一个硬件接口的时钟源（如同一个I2S）。由于时钟源与采样率相关，当录音和播放使用不同采样率时，会导致时钟源切换，从而使回采数据异常。因此在同一硬件通路上，录音和播放的采样率必须保持一致。如客户有特殊需求确实需要不同采样率，建议在上层（HAL或应用层）通过重采转换
2. 两种情况下播放的增益不一致，播放增益过小未达到回采触发电压会导致录不到对应的音频数据

4.4、回采声道获取到的音频数据听起来有杂音

具体表现为回采通道获取到规律脉冲杂音

可能原因：

1. 播放1khz正弦波，确认输出源是否干净
2. 如果是使用multi-dai情况下遇到的问题，可能是驱动原有的bug，请联系rk获取新的驱动

4.5、安卓平台命令录音正常，应用录音报错

具体表现为基于tinycap/tinyplay正常，使用app无法正常录音/指针不动

可能原因：

1. 一般是hal层适配问题，安卓平台的适配可根据《Rockchip_Developer_Guide_Android_3A_Module_CN》文档对照参考

4.6、音频通路打通后后续调试与适配步骤参考

1. 对于通用Linux平台，后续可基于alsa或rockit接口进一步做算法标定和算法适配、参数调优的工作，所需文档和算法库请联系rk获取，若使用rockit接口请参考[Rockchip_Developer_Guide_Linux_RV_Series_ACodec_CN.pdf](#)，vqe算法参考[Rockchip_Developer_Guide_Audio_Algorithm_VQE_Introduction_CN.pdf](#)
2. 对于Linux ipc类平台，需基于rockit接口进一步做算法标定和算法适配、参数调优的工作
3. 对于安卓平台，Android12-15平台可根据《Rockchip_Developer_Guide_Android_3A_Module_CN》文档进一步适配，Android11及更早的

平台请联系rk获取专门的补丁包